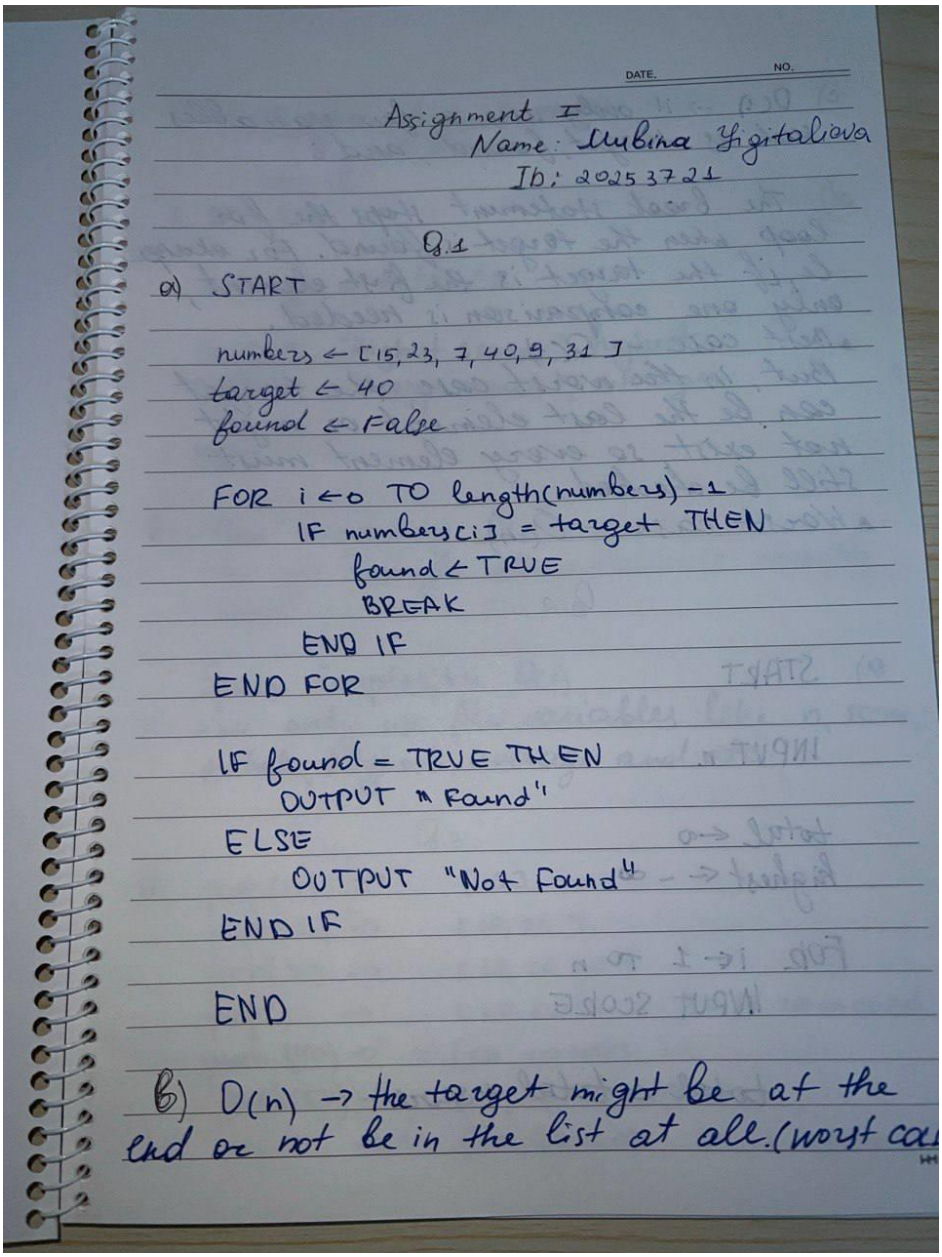
Data Structure Assignment1

Name: Yigitalieva Mubina
ID: 20253721



DATE. NO.

Assignment I
Name: Mubina Yigitalieva
Id: 20253721

Q.1

a) START

numbers ← [15, 23, 7, 40, 9, 31]
target ← 40
found ← False

FOR i ← 0 TO length(numbers) - 1
    IF numbers[i] = target THEN
        found ← TRUE
        BREAK
    END IF
END FOR

IF found = TRUE THEN
    OUTPUT "Found"
ELSE
    OUTPUT "Not Found"
END IF

END

B) O(n) -> the target might be at the end or not be in the list at all. (worst ca

c) O(1) → it only uses a few variables such as target, found, and i.

d) The break statement stops the for loop when the target is found. For example, if the target is the first element, only one comparison is needed.

- Best case - O(1).

But, in the worst case the target can be the last element or might not exist so every element must still be checked.

- Worst case - O(n)

Q.2.

a)

```
START

INPUT n

total ← 0
highest ← -∞

FOR i ← 1 TO n
    INPUT SCORE

    total ← total + score
```

DATE. NO.

```
    IF score > highest THEN
        highest ← score
    END IF
END FOR

average ← total / n

OUTPUT "Highest score:", highest
OUTPUT "Average score:", average

END
```

b) Time Complexity: O(n)
- we go through all n students exactly once.

Space Complexity: O(1)
- we only use few variables like n, score, total, highest, average and etc..

Q.3.

a)
push(10) →	[10]	
push(20) →	[10, 20]	
push(30) →	[10, 20, 30]	
pop() →	[10, 20]	30 is removed.
push(40) →	[10, 20, 40]	
push(50) →	[10, 20, 40, 50]	
pop() →	[10, 20, 40]	50 is removed.

b) 30 → first pop()
50 → second pop()

c) [10, 20, 40]

d)

push()	O(1)
pop()	O(1)
peek()	O(1)
search()	O(n)

1) push() - adds an element to the top
2) pop() - removes the top element
3) peek() - looks at the top element without removing it.
4) search() - In the worst case: we may need to check all elements.

e) A stack is suitable for Undo because it follows:
LIFO = Last in, First Out.

The most recent function should be undone first.

Example:
Action 1: Type "Dota"
Action 2: Type "Structures"
Action 3: Change color.

DATE. NO.

The stack contains:
[Type Data, Type Structure, change color].

If we press Undo, the most recent action:
change color
is removed first.

Therefore, stack function matches how Undo works.

Q.4

numbers = [10, 20, 30, 40, 50]

a)

1) [10, 20, 25, 30, 40, 50]
(0, 1, 2, 3, 4, 5)

2) [40, 20, 25, 30, 50]

3) [10, 20, 25, 30, 50, 60]

4) [10, 25, 30, 50, 60]

b) △ insert → O(n) - inserting inside requires shifting.
△ pop → O(n) - removing an element from inside may require shifting to the left.
△ append → O(1) - add at the end
△ remove → O(n) - requires shifting

c)

1. insert(2, 25) → 30, 40, 50 move right
2. pop(4) → 50 moves left.
3. append(60) → no element move
4. remove(40) → 25, 30, 50, 60 move left

d)

the most efficient : append(60)
time complexity : O(1)

Python lists are dynamic, and adding an element at the end usually does not require shifting existing elements.

Q. 5.

word = "AlgorithmDesign"
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

a) word[2:12:2] output: "gr+me"
b) word[-2:3:-3] output: "gehz"
c) word[1:-2:3] output: "lzhe"
d) word[-1:4:-2] output: [redacted] "niemt"

e) O(1), accessing by index is O(1)
f) O(n), checking if exists is O(n)
g) O(n), reversing the entire string, the programs needs to copy all n characters.